

MySQL 崩溃恢复过程分析

爱可生开源社区 2022-09-06 16:35 发表于上海

以下文章来源于一树一溪，作者操盛春



一树一溪

持续研究 MySQL 源码，致力于写出能让大家看明白的文章，欢迎关注，一起学习更多...

天有不测风云，数据库有旦夕祸福。

前面写 Redo 日志的文章介绍过，数据库正常运行时，Redo 日志就是个累赘。

现在，终于到了 Redo 日志扬眉吐气，大显身手的时候了。

本文我们一起来看看，MySQL 在崩溃恢复过程中都干了哪些事情，Redo 日志又是怎么大显身手的。

本文介绍的崩溃恢复过程，包含 **server 层** 和 **InnoDB**，不涉及其它存储引擎，内容基于 **MySQL 8.0.29** 源码。

目录

- 1. 概述
- 2. 读取两次写页面
- 3. 恢复数据页
 - 3.1 找到 last_checkpoint_lsn
 - 3.2 修复损坏的数据页
 - 3.3 读取 Redo 日志
 - 3.4 应用 Redo 日志
- 4. 删除 undo 表空间
- 5. 初始化事务子系统
- 6. 重建 undo 表空间
- 7. 处理事务
 - 7.1 清理已提交事务
 - 7.2 回滚未提交 DDL 事务
 - 7.3 回滚未提交 DML 事务
 - 7.4 处理 PREPARE 事务

- 8. 总结
- 9. 相关文章

正文

1. 概述

MySQL 崩溃也是一次关闭过程，只是比正常关闭着急了一些。

正常关闭时，MySQL 会做一系列收尾工作，例如：清理 undo 日志、合并 change buffer 缓冲区等操作。

具体会进行哪些收尾工作，取决于系统变量 `innodb_fast_shutdown` 的配置。

崩溃直接就是戛然而止，撂挑子不干了，还没来得及进行的那些收尾工作怎么办？

那就只能等待下次启动的时候再干了，这就是本文要介绍的 **崩溃恢复** 过程。

2. 读取两次写页面

MySQL 一旦崩溃，Redo 日志就要去拯救世界了（MySQL 就是它的世界），Redo 日志拯救世界的方式就是把还没来得及刷盘的脏页恢复到崩溃之前那一刻的状态。

虽然 Redo 日志能够用来恢复数据页，但这是有前提条件的：数据页必须完好无损的状态。

本文我们把系统表空间、独立表空间、undo 表空间中的页统称为 **数据页**。

如果数据页刚写了一半，MySQL 就戛然而止，这个数据页就损坏了，面对这种情况，Redo 日志也是巧妇难为无米之炊。

Redo 日志拯救世界之路就要因为这个问题停滞不前吗？

那显示是不能的，这就该轮到 **两次写** 上场了。

两次写 的官方名字是 **double write**，它包含 **内存缓冲区** 和 **dblwr 文件** 两个部分，InnoDB 脏页刷盘前，都会先把脏页写入内存缓冲区，再写入 dblwr 文件，成功之后才会把脏页刷盘。

两次写通过系统变量 `innodb_doublewrite` 控制开启或关闭，本文内容基于该系统变量的默认值 **ON**，表示开启两次写。

如果脏页写入内存缓冲区和 `dblwr` 文件的程中，MySQL 崩溃了，表空间中对应的数据页还是完整的，下次启动时，**不需要** 用两次写页面修复这个数据页。

如果脏页刷盘时，MySQL 崩溃了，表空间对应的数据页损坏了，下次启动时，应用 Redo 日志到数据页**之前**，**需要** 用两次写页面修复这个数据页。

dblwr 文件 默认位于 MySQL 数据目录下：

```
[csch@csch /usr/local/mysql_8_0_29/data] ls -l | grep dblwr
-rw-r----- 1 csch staff 192K 8 27 12:04 #ib_16384_0.dblwr
-rw-r----- 1 csch staff 8.2M 8 1 16:29 #ib_16384_1.dblwr
```

MySQL 启动过程中，会把 ***.dblwr** 文件中的所有两次写页面加载到两次写**内存缓冲区**，并用内存缓冲区中的两次写页面修复损坏的数据页，然后再应用 Redo 日志到数据页。

3. 恢复数据页

应用 Redo 日志到数据页（3.4 小节），需要先读取 Redo 日志（3.3 小节）。

读取日志 Redo 日志，需要有个起点，起点就是最后一次 checkpoint 的 lsn（3.1 小节）。

应用 Redo 日志有一个前提：数据页必须是**完好无损**的。要保证数据页的完整性，应用 Redo 日志之前需要修复损坏的数据页（3.2 小节）。

修复损坏数据页只需要保证在应用 Redo 日志之前就行了，之所以安排在 3.2 小节，是遵循了源码中的顺序。

了解本节安排内容顺序的逻辑，有助于理解应用 Redo 日志恢复数据页的过程，接下来我们正式进入下一个环节。

3.1 找到 last_checkpoint_lsn

读取 Redo 日志之前，必须先确定一个起点，这个起点就是 InnoDB 最后一次 checkpoint 操作的 lsn，也就是 **last_checkpoint_lsn**。

每个 Redo 日志文件的前 4 个 block 都是保留空间，不会用来写 Redo 日志，**last_checkpoint_lsn** 和其它 checkpoint 信息一起，位于第 1 个 Redo 日志文件的**第 2、4 个** block 中。

Redo 日志文件中每个 block 的大小为 512 字节。

InnoDB 每次进行 checkpoint 操作时，都会把 `checkpoint_no` 加 1，用于标识一次 checkpoint 操作。

然后把本次 checkpoint 信息写入 Redo 日志文件的第 2 或第 4 个 block 中。具体写入哪个 block，取决于 `checkpoint_no`。

如果 `checkpoint_no` 是奇数，checkpoint 信息写入 第 4 个 block。

如果 `checkpoint_no` 是偶数，checkpoint 信息写入 第 2 个 block。

确定读取 Redo 日志的起点时，从第 2、4 个 block 中读取较大的那个 `last_checkpoint_lsn` 作为起点。

为什么 checkpoint 信息要存储到 2 个 block 中？

这是一个用于保证 checkpoint 信息安全性的简单好用的方法，因为每次 checkpoint 只会往其中一个 block 写入信息。

万一就在某次写 checkpoint 信息的过程中 MySQL 崩溃了，有可能导致正在写入的这个 block 中的 checkpoint 信息不正确。

这种情况下，另一个 block 中的 checkpoint 信息肯定是正确的了，因为它里面的信息是上一次正常写入的。

能够用这种冗余方式来保证 checkpoint block 的安全性，基于一个前提：`last_checkpoint_lsn` 不需要那么精确。

`last_checkpoint_lsn` 比实际需要应用 Redo 日志起点处的 lsn 小是没关系的，不会造成数据页不正确，只是会多扫描一点 Redo 日志而已，应用 Redo 日志时会过滤已经刷盘的脏页对应的 Redo 日志。

3.2 修复损坏的数据页

把两次写文件中的所有数据页都加载到 内存缓冲区 之后，需要用这些页来把系统表空间、独立表空间、undo 表空间中损坏的数据页恢复到正常状态。

正常状态指的是 MySQL 崩溃之前，数据页最后一次 正确的 刷新到磁盘的状态。

恢复数据页的过程是对两次写 内存缓冲区 中的所有数据页进行循环，从 两次写数据页 中读取表空间 ID、页号，然后根据表空间 ID 和页号去系统表空间、独立表空间、undo 表空间中读取 对

应的数据页。

读取到 对应的数据页 之后，会根据其 File Header、File Trailer 中的一些字段判断数据页是不是已经损坏了：

首先，从 File Header 中读取 `FILE_PAGE_LSN` 字段，如果 `FILE_PAGE_LSN` 字段值大于当前系统已经生成的 Redo 日志的最大 LSN，说明数据库出现了不可描述的错误，数据页已经损坏。

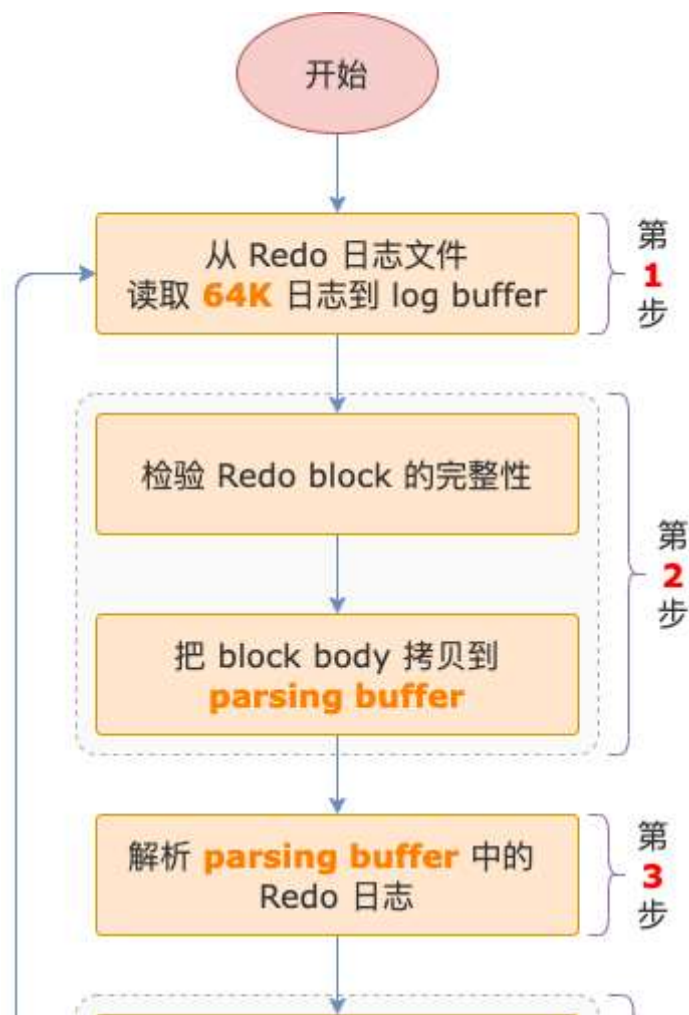
然后，从 File Header 中读取 `FILE_PAGE_SPACE_OR_CHECKSUM` 字段值，从 File Trailer 的前 4 字节中读取 checksum。

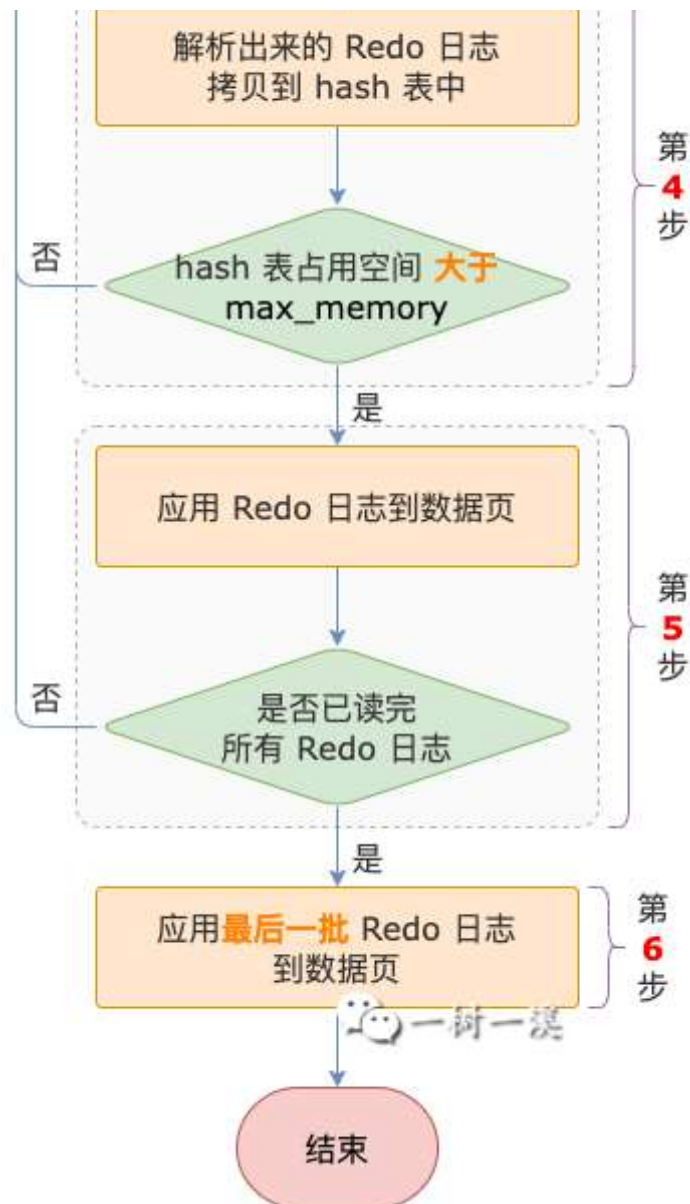
如果 `FILE_PAGE_SPACE_OR_CHECKSUM` 字段值和 File Trailer checksum 不一样，说明数据页已经损坏。

一旦出现了上面 2 种情况中的 1 种，把 两次写数据页 的内容 复制 到对应的数据页中，数据页就会恢复到正常状态了。

3.3 读取 Redo 日志

前面确定了读取 Redo 日志的起点 `last_checkpoint_lsn`，接下来就该读取 Redo 日志了，主要流程如下：





第 1 步，InnoDB 会以 **64K** 为单位，从 Redo 日志文件读取日志到 **log buffer** 中。

64K = 4 * innodb_page_size，所以，每次从 Redo 日志文件读取的数据量取决于系统变量 innodb_page_size。

第 2 步，已经读取到 log buffer 中的 block，利用 block header 和 block trailer 中的信息对 block 进行完整性检验之后，把 block body 信息拷贝到另一个缓冲区 **parsing buffer**。

parsing buffer 是一个 **2M** 的固定大小缓冲区，用于存放即将要被解析的 Redo 日志。

Redo 日志每个 block 的大小为 512 字节，block header 为 12 字节，block trailer 为 4 字节。

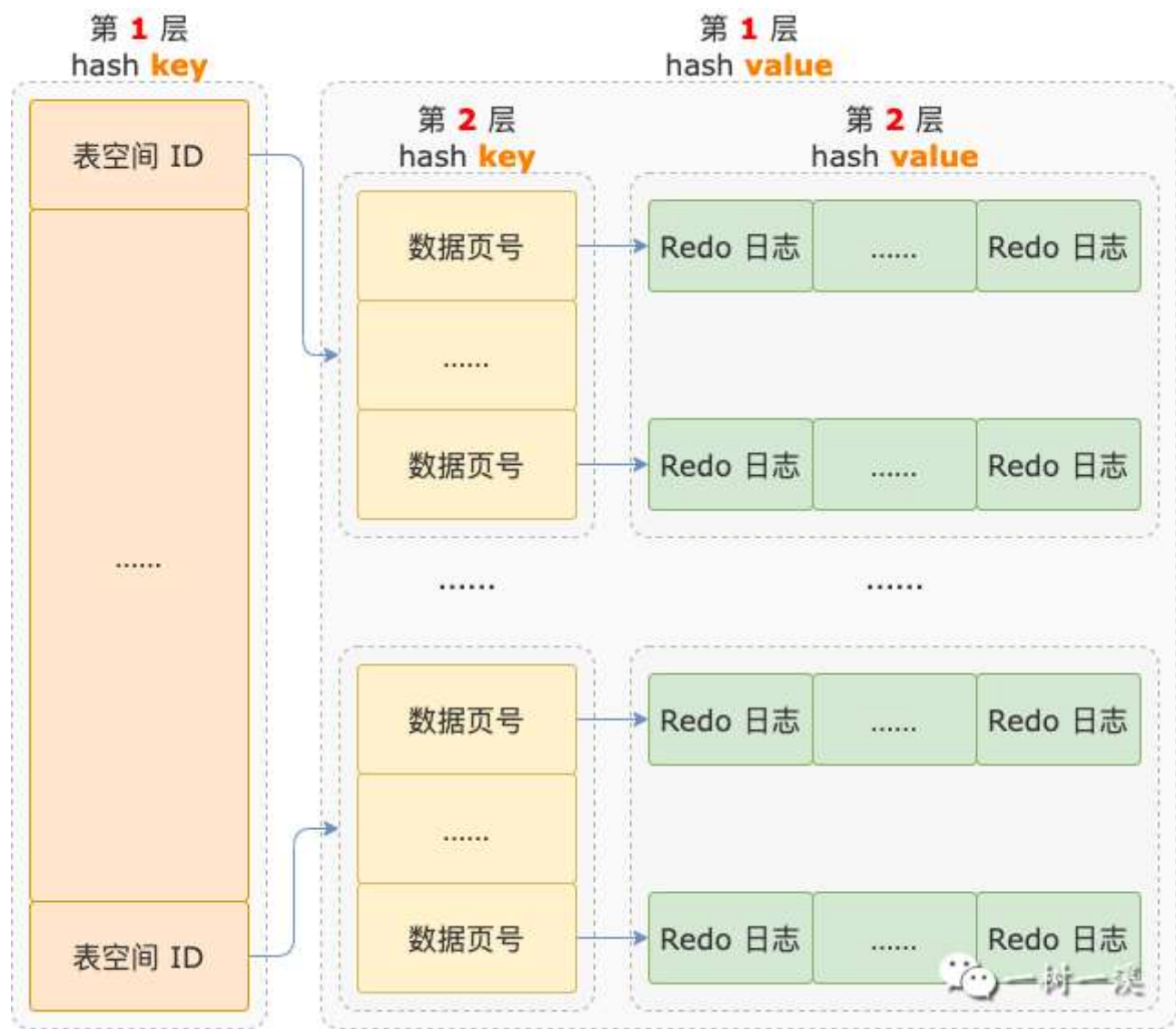
从 log buffer 的每个 block 中拷贝到 parsing buffer 的 block body 大小就是 $512 - 12 - 4 = 496$ 字节，也就是每个 block 中存放的 **Redo 日志数据** 部分。

第 3 步，解析 parsing buffer 中的 Redo 日志。

这一步解析 Redo 日志，实际上只是个预处理操作，并不会完整的解析每一条 Redo 日志，而是只会解析每一条 Redo 日志中的头信息以及数据地址，包括以 4 个部分：

- Redo 日志 类型
- Redo 日志所属数据页的 表空间 ID
- Redo 日志所属数据页的 页号
- Redo 日志 数据 ，这部分只是得到了每一条 Redo 日志在 block body 中的地址，后面应用 Redo 日志到数据页时会用到。

第 4 步，把第 3 步解析出来的每一条 Redo 日志的 4 个部分 都拷贝到 hash 表中。



这个 hash 表是个嵌套结构，第 1 层 hash key 是 表空间 ID ， value 也是个 hash 结构，也就是第 2 层。

同一个表空间的 Redo 日志以页单位组织到一起，存放到了以表空间 ID 为 key 的第 1 层 hash value 中。

第 2 层的 hash key 是 页号 ， value 是需要应用到这个数据页的 Redo 日志组成的链表。

同一个数据页的 Redo 日志链表以页号为 key，放在第 2 层 hash value 中。

链表中的 Redo 日志按照产生的先后顺序排列， 第 1 条 就是要应用的这些 Redo 日志中 最早 产生的那条。

第 5 步，应用 Redo 日志到数据页。

如果第 4 步进行的过程中，Redo 日志数据拷贝到 hash 表之后，导致 hash 表占用的空间 大于 max_memory，那么需要应用 Redo 日志到数据页，应用完成之后，清空 hash 表，为下一批 Redo 日志数据腾出空间。

这里的 max_memory 表示 hash 表能够使用的最大内存空间。

1 ~ 5 步是个循环执行过程，经过 N 轮循环之后，hash 表中有非常大的可能性还存在着最后一批 Redo 日志，因为占用空间 小于等于 max_memory 而只能在那里苦苦等待着被应用到 Redo 日志，这个工作就要等待第 6 步去干了。

第 6 步，收尾工作。

1 ~ 5 步循环结束之后，收尾工作就把 hash 表中剩下的 Redo 日志应用到数据页，这是崩溃过程中最后一次应用 Redo 日志。

前面都没有提到过存放 Redo 日志的 hash 表在哪里，能使用多大内存， 不知道你有没有好奇过？

这个 hash 表并不会单独申请一大块内存，而是借用了 buffer pool 中的内存。

因为在崩溃恢复过程中，进行到 读取 Redo 日志 阶段时，buffer pool 还没有真正开始用，所以可以先借来给 hash 表用一下。

不过 hash 表并不能使用 buffer pool 的全部内存，而是需要保留一部分内存，用于应用 Redo 日志到数据页的过程中，加载数据页到 buffer pool 中。

保留内存大小为：buffer pool 实例数量 * 256 个数据页，buffer pool 中的剩余内存，就是第 5 步提到的 max_memory，也就是 hash 表能够使用的最大内存。

3.4 应用 Redo 日志

前面介绍读取 Redo 日志，为了流程的完整性，有 2 个步骤已经涉及到应用 Redo 日志了。这里要介绍的是应用 Redo 日志的过程，会比上一小节深入一些。

读取 Redo 日志阶段，已经把所有需要应用的 Redo 日志都进行过预处理，并拷贝到 hash 表了。

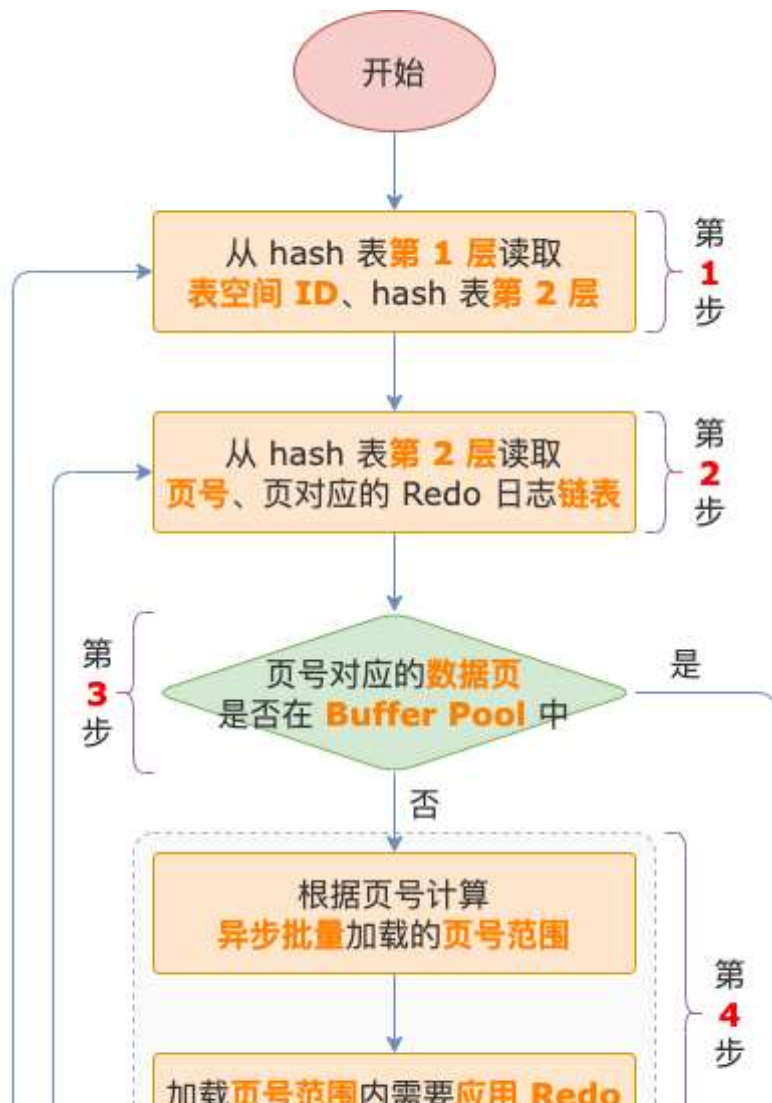
存放 Redo 日志的 hash 表是一个嵌套结构：

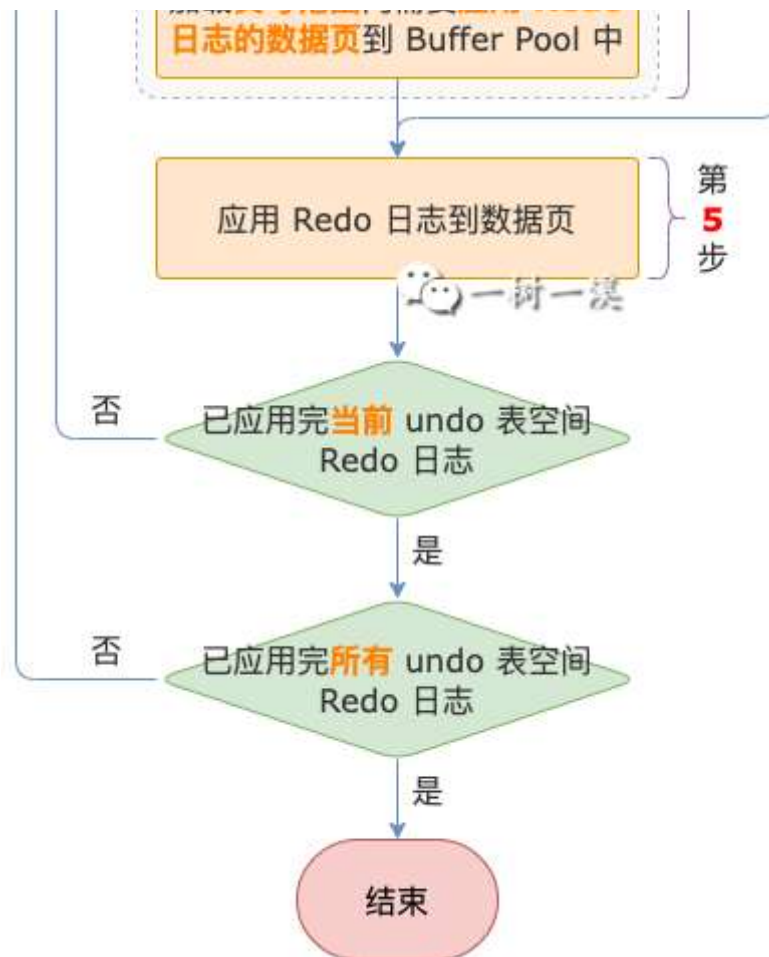
- 第 1 层的 hash key 是 表空间 ID ， hash value 还是一个 hash 表。
- 第 2 层的 hash key 是 页号 ， hash value 是个 Redo 日志链表，链表中的每个元素就是一条需要应用的 Redo 日志，按照产生的先后排序。

把每个数据页的 Redo 日志汇总到一起再去应用 Redo 日志，这样做的好处是效率高。

在崩溃恢复过程中，每个数据页只需要被加载到 buffer pool 中一次，一个数据页的 Redo 日志能够一次性应用，干脆利落。

应用 Redo 日志就是循环这个嵌套的 hash 表，把每一条 Redo 日志都应用到数据页中，主要流程如下：





第 1 步，从 **第 1 层** hash 表中取到 **表空间 ID** 和这个 undo 表空间下需要应用的 Redo 日志组成的第 2 层 hash 表。

第 2 步，从 **第 2 层** hash 表中取到一个 **页号** 和该数据页中需要应用的 Redo 日志链表。

第 3 步，判断当前循环的数据页是不是已经加载到 buffer pool 中了。

如果当前页 **没有** 加载到 buffer pool 中，进入第 4 步。

如果当前页 **已经** 加载到 buffer pool 中，进入第 5 步。

第 4 步，把不在 buffer pool 中的数据页加载到 buffer pool 中。

加载数据页到 buffer pool 中，是一个 **异步** 的 **批量** 操作，有可能会一次加载多个数据页。

也就是说，把数据页从表空间加载到 buffer pool 中会触发 **预读**，提前把一批需要应用 Redo 日志的数据页一次性加载到 buffer pool 中。

预读的数据页，不是随机读取的，而是根据第 3 步判断不在 buffer pool 中的数据页的页号（记为 **page_no**），计算出一个页号范围，把这个范围内需要应用 Redo 日志的数据页，全都加载到 buffer pool 中。

页号范围的起点: $\text{low_limit} = \text{page_no} - \text{page} \% 32$, 终点: $\text{low_limit} + 32$ 。

循环 $\text{low_limit} \sim \text{low_limit} + 32$ 范围内的页号, 只要碰到需要应用 Redo 日志的数据页, 就先把页号临时存放到一个数组里。

循环结束后, 把数组里的页号对应的数据页异步批量加载到 buffer pool 中。

从上面的逻辑可以看到, 一次预读最多只读 32 个数据页。

第 5 步, 应用 Redo 日志到数据页。

根据第 1 步取到的 表空间 ID 和第 2 步取到的 页号, 从 hash 表中获取该数据页需要应用的 Redo 日志链表。

从数据页的 File Header 中读取 **FILE_PAGE_LSN**, 循环 Redo 日志链表中的每一条日志, 判断该日志的 start_lsn 是否 大于等于 FILE_PAGE_LSN。

如果 $\text{start_lsn} < \text{FILE_PAGE_LSN}$, 说明该 Redo 日志对应的操作修改的数据页, 在 MySQL 崩溃之前就已经刷盘, 该 Redo 日志就不需要应用到数据页了。

如果 $\text{start_lsn} \geq \text{FILE_PAGE_LSN}$, 说明该 Redo 日志需要应用到数据页。

然后, 根据 Redo 日志类型, 调用不同的方法解析 Redo 日志, 直接修改 buffer pool 中的数据页, 对该数据页应用 Redo 日志的过程就完成了。

1 ~ 5 步是个循环过程, 直到所有 undo 表空间的 Redo 日志都被应用到数据页, 循环过程结束。

4. 删除 undo 表空间

MySQL 运行过程中, 如果有大事务往 undo 表空间中写入大量 undo 日志, undo 表空间会变大。

在早期版本中, undo 表空间变大之后, 就不能再缩回去了。

现在, 如果系统变量 **innodb_undo_log_truncate** 设置为 **on**, 当 undo 表空间增长到 **innodb_max_undo_log_size** 设置的大小 (默认值为 **1G**) 之后, InnoDB 会把这个 undo 表空间截断为初始大小 (16M)。

除了通过系统变量控制 undo 表空间自动截断之外, 还可以用下面这个 SQL 手动触发:

```
ALTER UNDO TABLESPACE tablespace_name
```

SET INACTIVE

不管自动还是手动，有可能 InnoDB 正在进行 undo 表空间截断操作，MySQL 就突然崩溃了，截断表空间操作还没有完成，那怎么办？

等到下次启动的时候，InnoDB 需要把未完成的 undo 表空间截断操作继续完成。

InnoDB 怎么知道哪些 undo 表空间的截断操作没有完成？

这就需要用到一个标记文件了，InnoDB 对某个 undo 表空间进行截断操作之前，会创建一个对应的标记文件，文件名是这样的： `undo_表空间编号_trunc.log`。

解释一下表空间的两个标识：表空间编号是给咱们人类看的，表空间 ID 是 MySQL 内部使用的，这两者不一样。

以 undo_001 表空间为例，表空间编号为 **1**，InnoDB 对 undo_001 表空间进行截断操作之前，会创建一个 `undo_1_trunc.log` 文件，如下：

```
[csch@csch /usr/local/mysql_8_0_29/data] ls -l | grep undo
-rw-r----- 1 csch staff 16M 8 27 12:04 undo_001
-rw-r----- 1 csch staff 16M 8 27 12:04 undo_002
-rw-r--r-- 1 csch staff 16K 6 22 12:36 undo_1_trunc.log
```

崩溃恢复过程中，InnoDB 如果发现某个表空间存在对应的 trunc.log 文件，说明这个 undo 表空间在 MySQL 崩溃时正在进行截断操作。

但是，只通过 trunc.log 文件存在这一个条件，并不能确定 undo 表空间截断操作没有完成，还要进一步判断。

接着读取 trunc.log 文件的内容，把读到的内容转换成数字，判断这个数字是不是等于 **76845412**。

76845412 是什么？稍候介绍。

如果 **等于**，说明在 MySQL 崩溃之前，undo 表空间截断操作 **已经** 完成，只是 trunc.log 文件还没来得及删除。此时，直接删除这个文件就可以了。

如果 **不等于**，说明 MySQL 崩溃时，undo 表空间截断操作 **还没有** 完成，那就需要继续完成。此时，直接删除 undo 表空间文件。

被删除的 undo 表空间要等到初始化事务子系统之后，才会重建，重建过程我们稍后介绍。

举个例子：启动过程中发现了 undo_001 表空间对应的 trunc.log 文件，并且文件中存储的数字不是 **76845412**，那就直接删除 undo_001 表空间。

删除之后，就只有 **undo_1_trunc.log** 文件能证明 undo_001 表空间存在过了，就像下面这样：

```
[csch@csch /usr/local/mysql_8_0_29/data] ls -l | grep undo
-rw-r----- 1 csch staff 16M 8 27 12:04 undo_002
-rw-r--r-- 1 csch staff 16K 6 22 12:36 undo_1_trunc.log
```

为什么这里不把 undo 表空间对应的 trunc.log 文件一起删除？

因为 undo 表空间要等到初始化事务子系统完成之后再重建，而 trunc.log 是 undo 表空间重建的凭证，所以，现在还不能删除。

接下来我们再看看 trunc.log 文件的创建和写入过程。

InnoDB 进行 undo 表空间截断操作之前，就会创建 trunc.log 文件（大小为 **innodb_page_size** 字节），并把文件内容的所有字节都初始化为 NULL，然后开始进行 undo 表空间截断操作。

操作完成之后，会往 trunc.log 文件中写入一个被称为 **魔数** 的数字：**76845412**，用于标识 undo 表空间截断操作已经完成。

如果魔数成功写入 trunc.log 文件，接下来会把 trunc.log 文件删除，undo 表空间的截断操作就结束了。

5. 初始化事务子系统

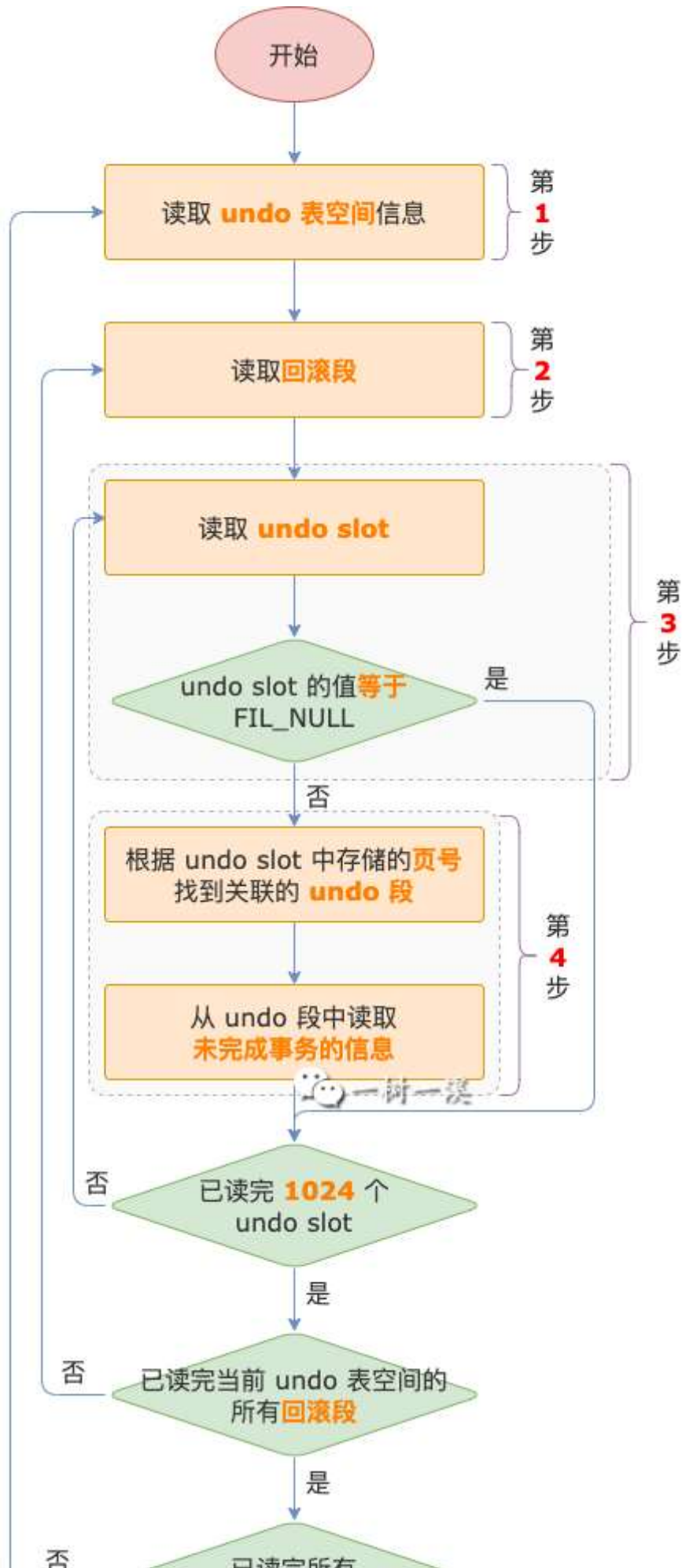
现在，我们来到了 **初始化事务子系统** 阶段。

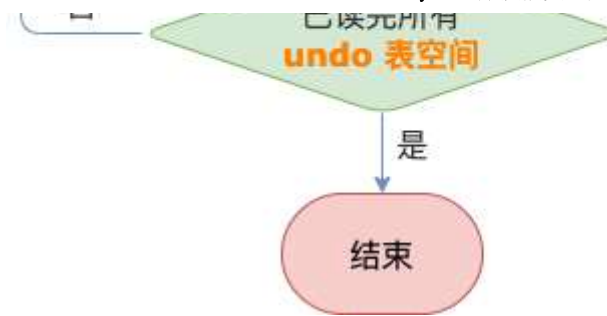
InnoDB 之所以把初始化事务子系统安排在删除 undo 表空间之后，有可能是为了 **避免** 读取要被删除的 undo 表空间，能够节省一点点时间。

删除还没有完成截断操作的 undo 表空间文件之后，剩下的 undo 表空间文件都需要读取。

从 undo 表空间文件读取未完成的事务，初始化事务子系统，主要过程如下：

初始化事务子系统还包含其它操作，不在本文介绍的范围内。





第 1 步，从内存中的 undo 表空间对象数组中读取 undo 表空间信息。

undo 表空间默认为 2 个，最多可以有 127 个。

有了独立 undo 表空间之后，位于系统表空间中的回滚段已经不再使用了，所以不需要从系统表空间的回滚段中读取事务信息。

第 2 步，从 undo 表空间中 页号 = 3 的数据页中读取回滚段。

每个 undo 表空间可以有 1 ~ 128 个回滚段，由系统变量 `innodb_rollback_segments` 控制，默认值为 2。

第 3 步，从回滚段中读取 undo slot。

回滚段的 段头页 中有 1024 个 undo slot（4 字节），每个 undo slot 对应一个 undo 段。

如果 undo slot 的值 等于 `FIL_NULL`，表示这个 undo slot 没有关联到 undo 段，继续执行第 3 步，读取下一个 undo slot。

如果 undo slot 的值 不等于 `FIL_NULL`，表示这个 undo slot 关联了 undo 段，进入第 4 步。

第 4 步，从 undo slot 对应的 undo 段中读取未完成事务的信息。

此时，undo slot 的值就是 undo 段的 段头页 的 页号，通过这个页号可以读取到 undo 段中的事务信息。

undo slot 关联了 undo 段，说明数据库崩溃时，undo 段中的事务还没有完成，事务状态可能是以下 3 种之一：

- `TRX_STATE_ACTIVE`，表示事务还没有进入提交阶段。
- `TRX_STATE_PREPARED`，表示事务已经提交了，但是只完成了二阶段提交的 `PREPARE` 阶段，还没有完成 `COMMIT` 阶段。

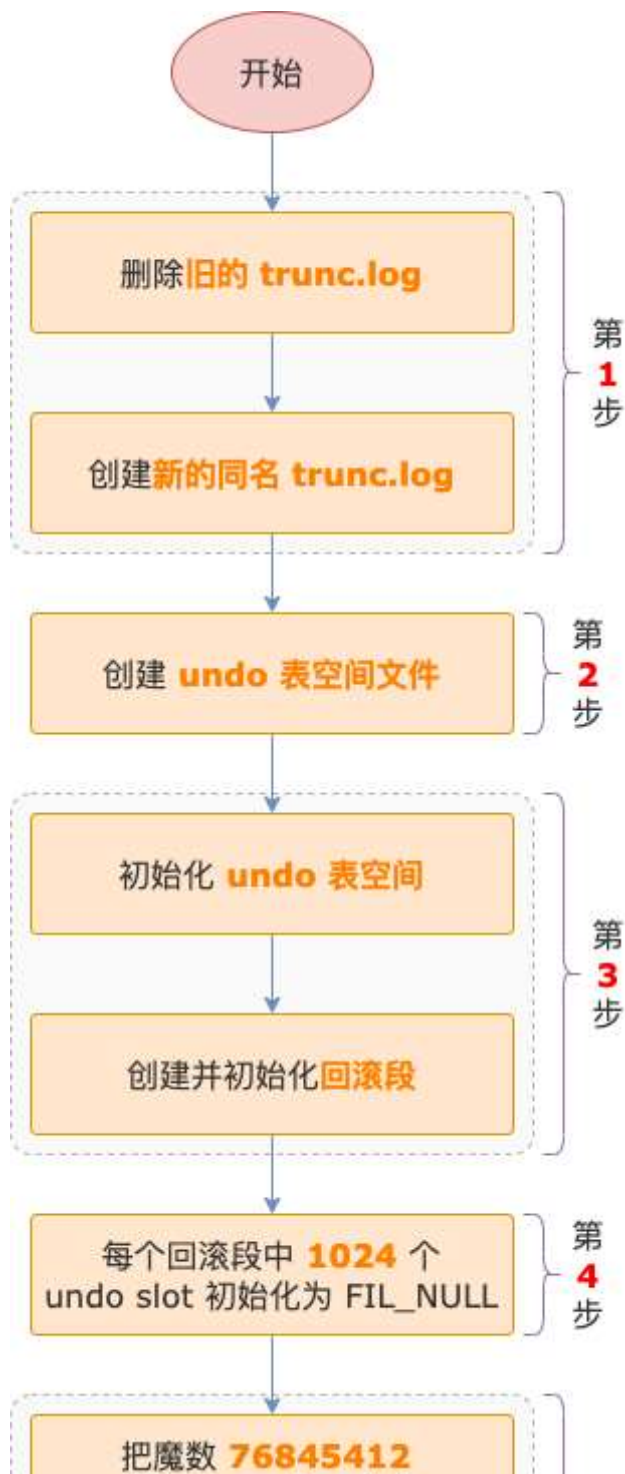
- **TRX_STATE_COMMITTED_IN_MEMORY**，表示事务已经完成了二阶段提交的 2 个阶段，还剩一些收尾工作没做，这种状态的事务修改的数据已经可以被其它事务看见了。

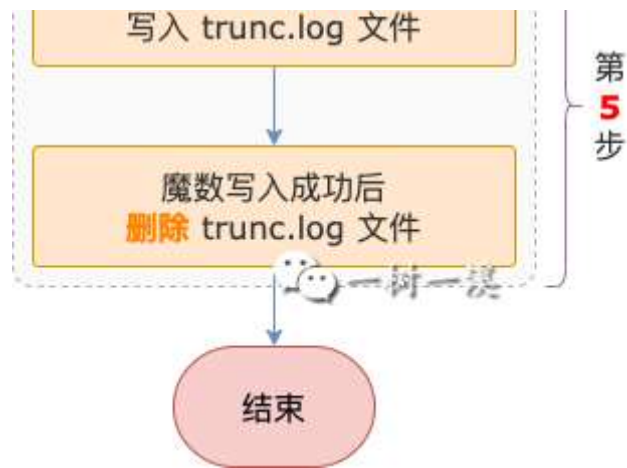
事务的收尾工作有哪些？**清理已提交事务** 小节会介绍。

第 1 ~ 4 步是个循环的过程，直到读完所有 undo 表空间中的事务信息结束。

6. 重建 undo 表空间

对于存在 trunc.log 文件的 undo 表空间，因为之前 undo 表空间文件被删除了，现在要开始着手重建 undo 表空间了，主要流程如下：





第 1 步，创建 trunc.log 文件，标记 undo 表空间重建操作正在进行中。

看到这里你可能会奇怪，undo 表空间对应的 trunc.log 文件不是没有删除吗？这里为什么又要创建一次？

别急，且往下看。

在创建 undo 表空间对应的 trunc.log 文件之前，会先删除之前 **旧的** trunc.log 文件，然后创建 **新的** trunc.log 文件。

新旧 trunc.log 文件名是一样的，例如：对于 undo_001 表空间来说，新旧 trunc.log 文件名都是 undo_1_trunc.log。

为什么要删除旧的 trunc.log 文件再创建新的同名 trunc.log 文件呢？

因为 **重建** undo 表空间和 **新建** undo 表空间是同一套逻辑，而新建 undo 表空间之前，该表空间并不存在对应的 trunc.log 文件。

为了保持统一的逻辑，所以会先删除已经存在的 trunc.log 文件。

第 2 步，创建 undo 表空间文件，初始大小为 **16M**，这个大小是硬编码的。

第 3 步，初始化 undo 表空间，把表空间 ID、各种链表信息写入表空间的 **0 号页** 中，然后分配一个新的数据页，创建并初始化回滚段，回滚段数量由系统变量 **innodb_rollback_segments** 控制。

第 4 步，循环 undo 表空间中的所有回滚段，把每个回滚段中的 1024 个 undo slot 都初始化为 **FIL_NULL**。

第 5 步，标记 undo 表空间重建操作已经完成。

InnoDB 会先往 trunc.log 文件中写入一个魔数 **76845412**，表示重建表空间操作已经完成。

写入魔数成功之后，再把 trunc.log 文件删除，重建一个 undo 表空间的过程就结束了。

如果有多个 undo 表空间需要重建，对于每个 undo 表空间都需要进行 1 ~ 5 步的流程。

7. 处理事务

在 **初始化事务子系统** 小节，我们介绍过，从 undo 表空间中读取出来的事务有 3 种状态：

- TRX_STATE_ACTIVE
- TRX_STATE_PREPARED
- TRX_STATE_COMMITTED_IN_MEMORY

处理事务阶段对这 3 种状态会进行不同的处理，请接着往下看。

7.1 清理已提交事务

这里要清理的已提交事务，指的是状态为 **TRX_STATE_COMMITTED_IN_MEMORY** 的事务，包含 DDL 和 DML 事务。

这种状态的事务已经 **完成** 二阶段提交的 PREPARE 和 COMMIT 阶段，是已经提交 **成功** 的事务，只差最后一点点清理工作，它们修改的数据已经 **能** 被其它事务看见了。

清理工作主要有几点：

- 处理 insert undo 段。
如果 insert undo 段 **能** 被缓存，undo 段会被加入 insert_undo_cached 链表尾部，以备重复使用；
如果 insert undo 段 **不能** 被缓存，undo 段就会被释放。
- 把事务从 **读写事务链表** 中删除。
- 把事务状态修改为 **TRX_STATE_NOT_STARTED**。

7.2 回滚未提交 DDL 事务

未提交事务指的是状态为 **TRX_STATE_ACTIVE** 的事务，也就是 **活跃** 事务。

崩溃恢复过程中，这种状态的事务是需要直接回滚的。

你可能会有个疑问，DDL 事务不是不能回滚吗？

DDL 事务不能回滚，这只是针对 MySQL 用户而言，MySQL 内部并不会受到这个限制。

我们在使用 MySQL 的过程中，如果在一个 DML 事务中间执行了一条 DDL 语句，会触发隐式提交，直接把 DML 事务提交了。

然后 DDL 会开启一个新事务，这个新事务是 **自动提交** 的，DDL 执行完成之后，事务就直接提交了，我们是没有机会对 DDL 事务进行回滚操作的。

MySQL 没给我们回滚 DDL 事务的机会，但是它自己有这个特权。

7.3 回滚未提交 DML 事务

未提交的 DDL 事务和 DML 事务在源码中是在不同时间触发的，它的回滚过程和 DDL 事务一样。

事务回滚的过程比较复杂，本文我们就不展开说了，后续会写一篇文章专门介绍事务回滚的过程。

7.4 处理 PREPARE 事务

PREPARE 事务指的是状态为 **TRX_STATE_PREPARED** 的事务，这种状态的事务比较特殊，在崩溃恢复过程中，既有可能被提交，也有可能被回滚。

PREPARE 事务提交还是回滚，取决于这个事务的 XID 是否已经写入到 binlog 日志文件中。

事务 XID 是以 binlog event 的方式写入 binlog 日志文件的，event 的名字是 **XID_EVENT**。

一个事务只会有一个 XID，也就只会有一个 XID_EVENT 了。

要知道事务的 XID_EVENT 是否已经写入到 binlog 日志文件，需要先读取 binlog 日志文件。

从上面的介绍可以看到，处理 PREPARE 事务依赖于 binlog 日志文件，因此，这部分逻辑是在 **打开** binlog 日志文件的过程中实现的。

MySQL 在同一时刻只会往一个 binlog 日志文件中写入 binlog event，在崩溃那一刻，承载写入 event 的文件是 **最后一个** binlog 日志文件。

因此，崩溃恢复过程中，只需要扫描最后一个 binlog 日志文件，找到其中所有的 XID_EVENT，用于判断 PREPARE 事务的 XID_EVENT 是否已经写入 binlog 日志文件。

如果 MySQL 上一次是 **正常关闭**，启动过程中，不会存在没有完成的事务，没有 PREPARE 事务需要处理，也就不需要扫描最后一个 binlog 日志文件了。

MySQL 怎么知道上一次是不是正常关闭呢？

每个 binlog 日志文件的第 1 个 EVENT 都是 **FORMAT_DESCRIPTION_EVENT**，用于描述 binlog 日志文件格式信息，这个 EVENT 中包含一个标记 **LOG_EVENT_BINLOG_IN_USE_F**。

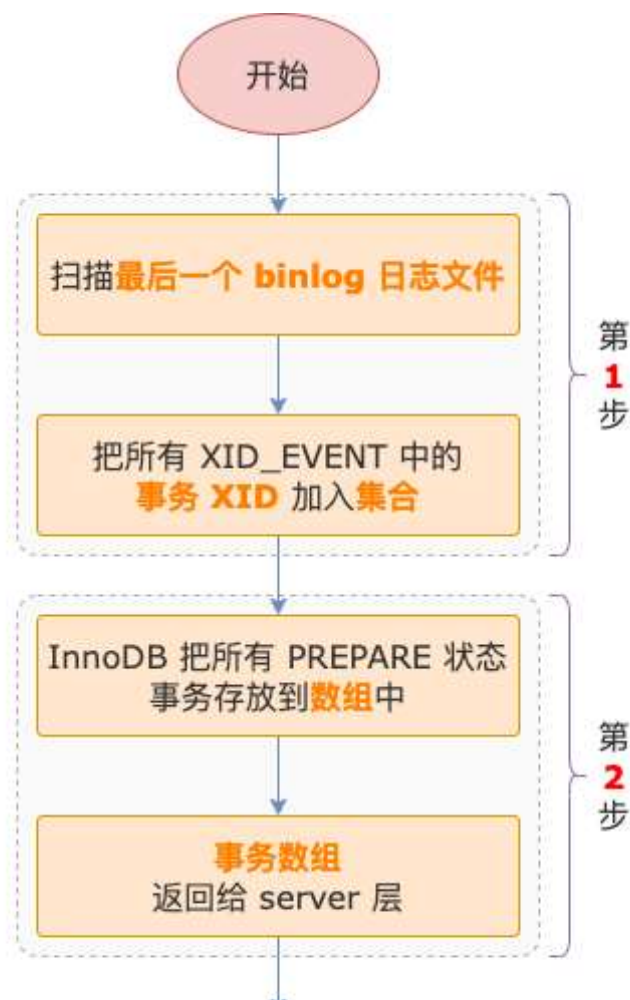
binlog 日志文件创建时，这个标记位会被设置为 **1**，表示 binlog 日志文件正在被使用。

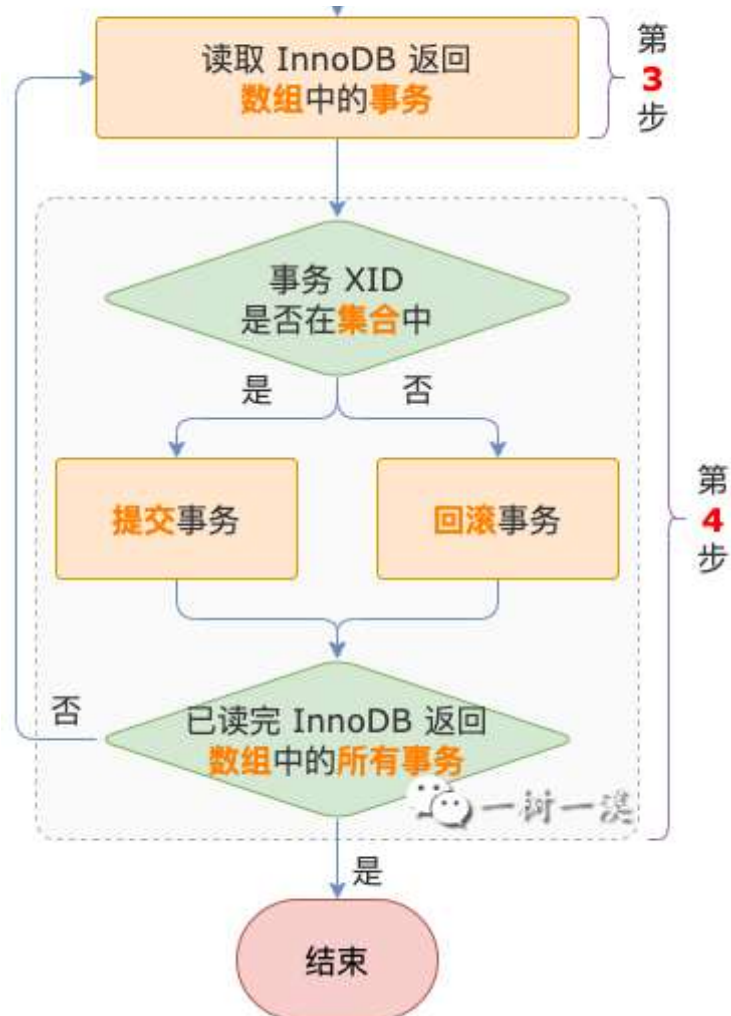
LOG_EVENT_BINLOG_IN_USE_F 标记在 2 种情况下会被清除：

- 切换 binlog 日志文件时，旧 binlog 日志文件的 **LOG_EVENT_BINLOG_IN_USE_F** 标记会被清除。
- MySQL 正常关闭时，正在使用的 binlog 日志文件的 **LOG_EVENT_BINLOG_IN_USE_F** 标记会被清除。

如果 MySQL 突然崩溃，来不及把这个标记设置为 0。

那么下次启动时，MySQL 读取最后一个 binlog 日志文件的 **FORMAT_DESCRIPTION_EVENT** 发现 **LOG_EVENT_BINLOG_IN_USE_F** 标记为 1，就会进入处理 PREPARE 事务阶段，主要流程如下：





第 1 步，扫描最后一个 binlog 日志文件，读取 EVENT，找到其中所有的 XID_EVENT，并把读取到的 事务 XID 存放到一个 集合 中。

第 2 步，InnoDB 循环 读写事务链表，每找到一个 PREPARE 事务都存放到 数组 中，最后把数组返回给 server 层。

第 3 步，读取 InnoDB 返回的 PREPARE 事务数组，判断事务 XID 是否在第 1 步的 事务 XID 集合 中。

第 4 步，提交或回滚事务。

如果事务 XID 在 集合中，说明 MySQL 崩溃之前，事务 XID_EVENT 就已经写入 binlog 日志文件了。

XID_EVENT 有可能已经同步给从服务器，从服务器上可能已经重放了这个事务。

这种情况下，为了保证主从数据的一致性，事务在主服务器上也 需要提交。

如果事务 XID 不在 集合中，说明 MySQL 崩溃之前，事务 XID_EVENT 没有写入 binlog 日志文件。

XID_EVENT 肯定也就没有同步给从服务器了，同样为了保证主从数据的一致性，事务在主服务器上也不能提交，而是需要回滚。

3 ~ 4 步是个循环过程，循环完 InnoDB 返回的 PREPARE 事务数组之后，处理 PREPARE 事务的过程结束，崩溃恢复主要流程也就完成了。

8. 总结

MySQL 崩溃恢复过程的核心工作有 2 点：

- 对于 MySQL 崩溃之前还没有刷新到磁盘的数据页（也就是脏页），用 Redo 日志把这些数据页恢复到 MySQL 崩溃之前那一刻的状态，这相当于对脏页进行一次刷盘操作。

在这之前，需要用两次写缓冲区中的页把损坏的数据页修复为正常状态，然后才能在此基础上用 Redo 日志恢复数据页。

- 清理、提交、回滚还没有完成的事务。

对于已完成二阶段提交的 PREPARE、COMMIT 2 个阶段的事务，做收尾工作。

对于活跃状态的事务，直接回滚。

对于 PREPARE 状态的事务，如果事务 XID 已写入 binlog 日志文件，提交事务，否则回滚事务。

9. 相关文章

- [Redo 日志从产生到写入日志文件](#)
- [Undo 日志用什么存储结构支持无锁并发写入？](#)
- [MySQL 事务二阶段提交](#)

以上就是本文的全部内容了，如果本文对你有所帮助，还请帮忙转发朋友圈、点赞、在看，谢谢 ^_^
有想了解的 MySQL 知识点或想交流的问题可以加我微信

公众号：一树一溪

我的微信：csch52



阅读原文

喜欢此内容的人还喜欢

太全了！用Python操作MySQL的使用教程集锦！
杰哥的IT之旅



MySQL性能调优，必须掌握这一个工具！！！（1分钟系列）
架构师之路



MySQL安装部署
IT那活儿

